

InsightFlow Agent 项目需求文档 v4

1. 项目名称

InsightFlow Agent: Multi-Agent Tool-Calling BI Workflow

中文名:

基于 LangGraph 的多智能体工具调用式业务数据分析工作流

2. 一句话介绍

InsightFlow Agent 是一个面向业务数据分析场景的多智能体工具调用系统。

用户输入自然语言业务问题后，系统通过多个专业 Agent 协作，调用真实工具完成数据库 Schema 获取、业务指标定义检索、SQL 生成、SQL 安全校验、SQL 执行、错误修复、数据解释、图表生成、报告保存、Trace 记录和 AgentEval 评测。

项目第一目标不是做一个普通 ChatBI 或 Text2SQL，而是突出：

```text id="7lreq7" Multi-Agent 协作 Tool Calling SQL Execution Execution Feedback Repair Trace Eval

---

### ## 3. 项目定位

本项目是你现有 Agentic RAG 项目的互补项目。

现有 RAG 项目重点证明：

```text id="d7w7x4"

非结构化文档检索

RAG workflow

引用校验

答案可靠性评估

InsightFlow 重点证明：

```text id="bo0kl8" 结构化数据分析 数据库工具调用 SQL 生成与校验 SQL 真实执行 SQL 错误自动修复 业务指标口径校验 图表与报告生成 Trace / Eval 后续行动型 workflow

因此，本项目不能做成“又一个 RAG 问答项目”，也不能只做成“Text2SQL + 图表”。

本项目最终要表达的是：

```
```text id="ub94v6"
```

Agent 不是直接回答问题，而是根据任务目标调用真实工具；
工具返回真实结果后，Agent 再根据结果决定下一步；
整个过程由 LangGraph State 和条件边控制，并且可追踪、可评测。

4. 项目核心目标

4.1 P0 核心目标

P0 只做最小但扎实的 Agentic SQL Core。

目标是证明：

```
```text id="9hqwkl" 这不是普通 Text2SQL，而是一个 Multi-Agent Tool-Calling SQL Workflow。
```

P0 必须支持：

```
```text id="axw5gb"
```

自然语言业务问题输入
数据库 Schema 读取
业务指标定义读取
SQL 生成
SQL 安全校验
SQL 真实执行
SQL 执行失败后自动修复一次
自然语言结果解释
完整 Trace 记录
基础 AgentEval
Streamlit Demo

4.2 P1 核心目标

P1 加入可靠分析和报告交付能力。

目标是证明：

```
```text id="oztgyt" 系统不只是会查 SQL，还能生成可追溯的业务分析产物。
```

P1 增加：

```
```text id="u67gc2"
```

Business Context RAG 简化版
Evidence Validator
Data-supported findings vs hypotheses
Chart Agent

```
generate_chart()
Report Agent
save_report()
Markdown 分析报告
report completeness eval
unsupported claim eval
```

4.3 P2 核心目标

P2 加入经营周报和行动型工具调用能力。

目标是证明：

```text id="pk9p4v" 系统可以从数据分析走向业务行动，但仍然保留审批、安全和审计边界。

P2 增加：

```
```text id="3apy8a"
Business Review Report
经营周报 / 品类复盘 / 渠道复盘
Action Planner
Risk Assessor
Approval Gate
create_task()
create_metric_alert()
create_email_draft()
Action Verifier
Audit Log
ActionEval
```

4.4 P3 核心目标

P3 做工程化和工具标准化。

目标是证明：

```text id="jhjhf5" 项目具备接近真实生产环境的工具层、任务管理和部署能力。

P3 增加：

```
```text id="xzxtzb"
MCP Tool Layer
FastAPI Run API
Async Job
Trace Dashboard
RBAC
Docker
CI
```

Postgres
多数据源

5. 项目不做什么

本项目不是：

```text id="tm738m" 普通 Text2SQL Demo 普通 ChatBI 普通报表系统 普通周报生成器 任务管理系统 完整企业 BI 平台 又一个文档 RAG 项目

弱版本 Text2SQL：

```text id="2nun1j"  
用户问题
-> LLM 生成 SQL
-> 返回 SQL 文本

InsightFlow 目标：

```text id="fqg1bu" 用户问题 -> Supervisor Agent 判断任务 -> Schema Agent 调用 Schema Tool -> Metric Agent 调用 Metric Tool -> SQL Generator Agent 生成 SQL -> SQL Reviewer Agent 调用 validate\_sql -> SQL Executor 调用 run\_sql -> 如果失败，Error Fix Agent 修复 SQL -> 重新 validate\_sql -> 重新 run\_sql -> Insight Agent 基于结果回答 -> Trace Logger 记录全过程 -> Eval 统计成功率和失败类型

---

## 6. 业务场景选择

### 6.1 MVP 业务场景

MVP 使用 \*\*电商业务数据分析场景\*\*。

原因：

```text id="0ywej4"  
业务容易理解
数据表结构清晰
适合多表 JOIN
适合展示销售额、订单量、客单价、品类、商品、城市等分析
适合展示 SQL 错误修复
适合后续扩展图表、报告、周报和行动任务

6.2 支持的业务问题

P0 支持：

```text id="9e2vb9" 最近 30 天销售额最高的 5 个商品是什么？最近 3 个月销售额最高的品类是什么？每个城市的总销售额是多少？最近 6 个月每个月的订单量是多少？最近 3 个月销售额下降最多的品类有哪些？

P1 支持：

```text id="ykk4s5"

最近 3 个月销售额下降最多的品类有哪些？生成趋势图和分析报告。

最近 6 个月哪个品类销售额波动最大？

哪些商品销量异常下降？

P2 支持：

```text id="s7dfs" 帮我生成一份本周电商经营分析周报。帮我生成一份品类销售复盘报告。找出最近销售额下降最多的品类，并为运营团队创建跟进任务。生成本周经营周报，并创建需要下周复查的指标告警。

---

## 7. 数据库设计

### 7.1 P0 核心表

P0 使用 SQLite，包含 5 张核心表：

```text id="qwnnuk"

users

orders

order_items

products

categories

7.2 P0 表结构

users

```text id="6zg7qh" id name city gender age created\_at

#### orders

```text id="6ob28u"

id

user_id

order_date

status

total_amount

order_items

```
```text id="vygrrh" id order_id product_id quantity unit_price
```

```
products

```text id="4it5w6"
id
product_name
category_id
price
created_at
```

categories

```
```text id="87tezd" id category_name
```

```
7.3 P1 扩展表

```text id="kjt9f"
marketing_channels
```

用于支持渠道 ROI、渠道表现、周报中的渠道模块。

7.4 P2 扩展表

```
```text id="p406z9" analysis_runs reports tasks metric_alerts approvals audit_logs scheduled_runs
```

用于保存报告、任务、告警、审批、审计和定时复查记录。

---

## 8. 业务指标定义

### 8.1 指标定义文件

P0 使用 `data/metrics.yaml` 保存核心指标定义。

示例：

```
```yaml id="lp8fr9"
gmV:
  name: "销售额"
  formula: "SUM(order_items.quantity * order_items.unit_price)"
  required_filters:
    - "orders.status = 'paid'"
  exclude_status:
```

```

- "cancelled"
- "refunded"

order_count:
  name: "订单量"
  formula: "COUNT(DISTINCT orders.id)"
  required_filters:
    - "orders.status = 'paid'"

aov:
  name: "客单价"
  formula: "SUM(order_items.quantity * order_items.unit_price) / COUNT(DISTINCT orders.id)"
  required_filters:
    - "orders.status = 'paid'"

```

8.2 指标定义作用

Metric Tool / Metric Agent 用于:

```text id="456vz1" 帮助 SQL Generator 生成符合业务口径的 SQL 帮助 SQL Reviewer 检查 SQL 是否漏掉必要过滤条件 减少 “销售额到底怎么算” 的不一致问题

```

9. Agent / Tool 边界

9.1 Agent 负责决策

Agent 做:

```text id="h5ypun"
理解任务
判断任务类型
选择工具
生成 SQL
判断 SQL 是否合理
根据错误修复 SQL
解释执行结果
判断是否需要图表
判断是否需要报告
判断是否需要行动计划

```

9.2 Tool 负责执行

Tool 做:

```text id="c4i0ab" 读取数据库 Schema 读取指标定义 校验 SQL 执行 SQL 生成图表 保存报告 创建任务 创建告警 保存 Trace 保存 Audit Log

### ### 9.3 核心原则

```text id="keymk3"

Agent 不直接假设外部状态。

Agent 必须通过 Tool 获取真实结果。

Tool 返回结果写入 LangGraph State。

LangGraph 条件边根据 State 决定下一步。

10. Agent 设计

10.1 P0 Agent

| Agent | 是否必须 | 职责 |
|---------------------|------|---|
| Supervisor Agent | 建议 | 判断任务类型，初始化 workflow |
| Schema Agent | 必须 | 调用 <code>get_database_schema()</code> 获取数据库结构 |
| Metric Agent | 必须 | 调用 <code>retrieve_metric_definition()</code> 获取业务指标定义 |
| SQL Generator Agent | 必须 | 根据问题、Schema、指标定义生成 SQL |
| SQL Reviewer Agent | 必须 | 调用 <code>validate_sql()</code> 校验 SQL |
| Error Fix Agent | 必须 | 根据数据库错误和 Schema 修复 SQL |
| Insight Agent | 必须 | 基于 SQL 执行结果生成回答 |

10.2 P1 Agent

| Agent | 职责 |
|--------------------------|---|
| Context Retrieval Agent | 调用 <code>retrieve_business_context()</code> 检索业务规则、字段说明、历史 SQL 示例 |
| Evidence Validator Agent | 区分数据支持结论和假设 |
| Chart Agent | 判断图表类型并调用 <code>generate_chart()</code> |
| Report Agent | 组织 SQL、结果、图表、结论并调用 <code>save_report()</code> |

10.3 P2 Agent

| Agent | 职责 |
|-------------------------|-----------------|
| Report Supervisor Agent | 拆解经营周报 / 复盘报告任务 |
| Action Planner Agent | 根据分析结果生成行动计划 |
| Risk Assessor Agent | 判断动作风险 |
| Approval Gate | 等待用户审批 |

| Agent | 职责 |
|-----------------------|--------------------|
| Action Verifier Agent | 验证任务、告警、邮件草稿是否创建成功 |

11. Tool 设计

11.1 P0 必须工具

```
```text id="s5n1nk" get_database_schema() retrieve_metric_definition() validate_sql() run_sql()
log_trace()
```

```
11.2 P0 可选工具
```

```
```text id="58ghpc"
get_sample_rows()
```

11.3 P1 工具

```
```text id="n199qz" retrieve_business_context() validate_evidence() run_python_analysis()
generate_chart() save_report()
```

```
11.4 P2 工具
```

```
```text id="nfjjmu"
create_task()
create_metric_alert()
create_email_draft()
verify_action_execution()
log_audit_event()
schedule_followup_run()
```

11.5 P3 工具与服务

```
```text id="w5j6gp" database-mcp-server report-mcp-server action-mcp-server FastAPI run API async
job queue Trace Dashboard RBAC service
```

```

```

```
12. P0 Workflow: Agentic SQL Core
```

```
12.1 主流程
```

```
```text id="j5cxgg"
START
↓
```

```

Supervisor Agent
↓
Schema Agent -> get_database_schema()
↓
Metric Agent -> retrieve_metric_definition()
↓
SQL Generator Agent
↓
SQL Reviewer Agent -> validate_sql()
↓
SQL Executor Tool -> run_sql()
↓
Insight Agent
↓
Trace Logger -> log_trace()
↓
END

```

12.2 SQL 失败修复流程

```text id="evbwsz" run\_sql() failed ↓ Error Fix Agent ↓ SQL Reviewer Agent -> validate\_sql() ↓ SQL Executor Tool -> run\_sql() ↓ Insight Agent or Fail Response

### ### 12.3 条件边

```

```text id="zxzqba"
SQL Reviewer:
approved = true -> SQL Executor
approved = false -> SQL Generator

SQL Executor:
success = true -> Insight Agent
success = false and retry_count < 1 -> Error Fix Agent
success = false and retry_count >= 1 -> Fail Response

```

12.4 P0 限制

```text id="9nzrod" 自动修复最多 1 次 不允许无限循环 不允许执行非 SELECT SQL 失败时必须返回明确失败原因 最终回答必须基于 run\_sql 返回结果

```

13. P0 功能需求

13.1 用户输入

```

用户可以输入中文业务问题，例如：

```
```text id="pd4bdz"
最近 30 天销售额最高的 5 个商品是什么？
```

验收标准：

```
```text id="192a86" 前端可以接收中文问题 系统创建 run_id 系统开始 LangGraph workflow 前端展示执行状态
```

```

13.2 Schema Tool

工具名称：

```text id="v8zxpK"
get_database_schema()
```

功能：

```
```text id="w22oak" 读取 SQLite 数据库中的表、字段、类型、主键、外键。
```

输出示例：

```
```json id="dqqxko"
{
  "success": true,
  "tables": [
    {
      "table_name": "orders",
      "columns": [
        {"name": "id", "type": "INTEGER"},
        {"name": "user_id", "type": "INTEGER"},
        {"name": "order_date", "type": "TEXT"},
        {"name": "status", "type": "TEXT"},
        {"name": "total_amount", "type": "REAL"}
      ]
    }
  ]
}
```

验收标准：

```
```text id="wz0iu8" Agent 可以在生成 SQL 前获取数据库结构 Schema 写入 LangGraph State Schema 调用写入 Trace
```

---

### ### 13.3 Metric Tool

工具名称:

```
```text id="6px7k2"
retrieve_metric_definition()
```

功能:

```text id="eolcw5" 根据用户问题中的业务指标，返回 GMV、订单量、客单价等指标定义。

输入示例:

```
```json id="2s1zim"
{
  "question": "最近 30 天销售额最高的 5 个商品是什么?"
}
```

输出示例:

```
```json id="k651lx" { "metric": "gm", "formula": "SUM(order_items.quantity * order_items.unit_price)",
"required_filters": ["orders.status = 'paid'"] }
```

验收标准:

```
```text id="xgtt5q"
用户问“销售额”时能匹配 GMV 定义
SQL Generator 使用指标定义生成 SQL
SQL Reviewer 能检查 SQL 是否包含必要指标口径
```

13.4 SQL Generator Agent

功能:

```text id="wa3r4a" 根据用户问题、数据库 Schema 和指标定义生成候选 SQL。

输出格式:

```
```json id="3h02gl"
{
  "sql": "SELECT ...",
  "tables": ["orders", "order_items", "products"],
```

```
"metrics": ["gm"],
"reason": "需要通过订单表和订单明细表计算商品销售额。"
}
```

验收标准：

``text id="2t1nlk" 只能生成 SELECT SQL 默认包含 LIMIT 能够解释表选择原因 SQL 输出为结构化 JSON

```
---

### 13.5 SQL Reviewer Agent / validate_sql()

功能：

``text id="2jhvyg"
在执行 SQL 前校验安全性、结构正确性和业务指标口径。
```

检查内容：

``text id="sqzsq" 是否只包含 SELECT 是否包含 DROP / DELETE / UPDATE / INSERT / ALTER / TRUNCATE
是否包含多语句 是否包含 LIMIT 表名是否存在 字段名是否存在 JOIN key 是否合理 时间范围是否符合用户问题
销售额是否使用正确 GMV 口径 是否包含 orders.status = 'paid' 是否访问敏感字段

输出示例：

```
``json id="bm7t54"
{
  "approved": true,
  "risk_level": "low",
  "issues": [],
  "checks": {
    "select_only": true,
    "tables_exist": true,
    "columns_exist": true,
    "has_limit": true,
    "metric_formula_correct": true,
    "paid_filter_included": true
  }
}
```

验收标准：

``text id="8srfzq" 危险 SQL 必须被拒绝 非 SELECT SQL 不允许执行 字段不存在时能识别 指标口径错误时能
提示 校验失败时不进入 run_sql

13.6 SQL Executor / run_sql()

工具名称:

```
```text id="nw3vow"
run_sql()
```

功能:

```text id="ugiebx" 真实连接 SQLite 数据库并执行 SQL 查询。

约束:

```
```text id="5hc6fe"
只允许 SELECT
最大返回 100 行
支持 timeout
捕获数据库错误
返回结构化结果
```

成功输出:

```
```json id="015zI8" { "success": true, "columns": ["product_name", "sales"], "rows": [ ["Camera A",
128000], ["Lens B", 113500] ], "row_count": 2, "execution_time_ms": 34 }
```

失败输出:

```
```json id="xj9xlu"
{
 "success": false,
 "error": "no such column: oi.price",
 "execution_time_ms": 11
}
```

验收标准:

```text id="ddfltk" SQL 必须真实执行 成功时返回 rows 失败时返回 error 失败不允许中断 workflow 执行结果写入 State 和 Trace

13.7 Error Fix Agent

功能：

```
```text id="gp6j4a"
```

当 SQL 执行失败时，根据原 SQL、数据库错误、Schema 和用户问题生成修复 SQL。

典型修复场景：

```
```text id="airdw1" 字段不存在 表名错误 SQL 语法错误 JOIN 字段错误 时间字段错误
```

示例：

```
```text id="jtyoa4"
```

错误：no such column: oi.price

修复：order\_items 表中没有 price 字段，应使用 unit\_price

验收标准：

```text id="vs7on3" 执行失败后能进入 Error Fix Agent 修复后的 SQL 必须重新经过 validate\_sql 修复后再次调用 run\_sql 最多自动修复 1 次 仍失败时返回失败原因，不编造答案

13.8 Insight Agent

功能：

```
```text id="jgub29"
```

根据 run\_sql 返回结果生成自然语言结论。

要求：

```text id="ky58ma" 结论必须基于 execution\_result 必须包含关键数值 查询结果为空时说明无数据 SQL 失败时说明失败原因 不允许编造数据库不存在的信息

13.9 Trace Logger

工具名称：

```
```text id="c4a154"
```

log\_trace()

记录内容：

```
```text id="mpc4c3" run_id session_id user_question node tool_name tool_input
tool_output_summary status latency_ms error_type retry_count final_answer timestamp
```

验收标准：

```
```text id="8gxl9m"
每次 run 都生成 trace.json
每次工具调用都进入 Trace
Trace 可以用于复盘 Agent 执行路径
Trace 不保存敏感明文数据
```

## 14. P1 功能需求：可靠分析与报告生成

P1 的重点不是做完整“周报系统”，而是先做 **可追溯的分析报告生成能力**。

### 14.1 Business Context RAG 简化版

工具名称：

```
```text id="lu6gqi" retrieve_business_context()
```

检索内容：

```
```text id="8obwxo"
指标定义
业务规则
字段说明
历史 SQL 示例
报告模板
```

实现方式：

```
```text id="6wj2n5" YAML / JSON / Markdown + keyword matching
```

暂不强制使用向量数据库。

14.2 Evidence Validator Agent

功能：

```
```text id="n42kfk"
区分数据支持结论和需要进一步验证的假设，防止 Agent 编造业务原因。
```



输出示例：

```
```json id="xr60gr" { "data_supported_findings": [ { "claim": "Camera 品类 GMV 下降 32.8%", "evidence": "SQL result: gmv_drop_rate = 0.328", "confidence": 0.95 } ], "hypotheses": [ { "claim": "可能与广告流量下降有关", "reason": "当前数据库没有曝光、点击率和转化率数据，无法验证", "needs_more_data": [ "ad_impressions", "ctr", "conversion_rate" ] }, { "claim": "库存不足是导致销量下降的主要原因" } ] }
```

验收标准：

```
```text id="7i5mfp"
```

报告中必须区分 findings 和 hypotheses

没有数据支持的原因不能写成确定结论

Eval 中统计 unsupported\_claim\_rate

---

### 14.3 Chart Agent / generate\_chart()

功能：

```
```text id="huxrcf" 根据查询结果和任务类型生成图表。
```

推荐规则：

```
```text id="je1q5c"
```

排名问题 -> bar chart

趋势问题 -> line chart

占比问题 -> pie chart, 可选

下降分析 -> line chart + bar chart

输出：

```
```json id="izu8m" { "success": true, "chart_path": "reports/charts/run_001_category_gmv_trend.png" }
```

验收标准：

```
```text id="83f3ro"
```

图表文件真实生成

图表路径写入 State

图表生成工具调用进入 Trace

---

### 14.4 Report Agent / save\_report()

功能：

```text id="4txwtj" 将 SQL、查询结果、图表、数据支持结论、假设和建议保存为 Markdown 分析报告。

报告结构：

```text id="fq97s7"

1. 用户问题
2. 使用的业务指标
3. 执行 SQL
4. 查询结果摘要
5. 数据支持结论
6. 需要进一步验证的假设
7. 图表路径
8. 下一步建议
9. Trace 路径

输出：

```json id="juw4zj" { "success": true, "report\_path": "reports/markdown/run\_001\_decline\_analysis.md" }

验收标准：

```text id="fda5mb"

报告文件真实保存  
报告中的结论能追溯到 SQL 和 execution\_result  
报告包含 Trace 路径  
报告不包含未验证的确定性原因

## 15. P2 功能需求：经营周报与行动闭环

P2 才做周报，不放进 P0。

### 15.1 Business Review Report

功能：

```text id="ybuhiy" 支持生成经营周报、品类复盘、渠道复盘等多模块报告。

示例输入：

```text id="shw746"

帮我生成一份本周电商经营分析周报，包括销售额、订单量、Top 商品、下降品类、渠道表现和运营建议。

Supervisor 拆解为：

```text id="g0e5t2" 本周 GMV 本周订单量 本周客单价 Top 5 商品 Top 5 品类 下降最多品类 渠道表现 异常点  
下周建议

15.2 周报生成 Workflow

```
```text id="c00bx1"
Report Supervisor Agent
↓
Schema Agent -> get_database_schema()
↓
Metric Agent -> retrieve_metric_definition()
↓
SQL Generator Agent -> 为多个模块生成 SQL
↓
SQL Reviewer Agent -> validate_sql()
↓
SQL Executor -> run_sql()
↓
Python Analyst -> run_python_analysis()
↓
Evidence Validator
↓
Chart Agent -> generate_chart()
↓
Report Agent -> save_report()
```

## 15.3 周报输出

```text id="c6u13k" reports/markdown/weekly\_business\_report.md reports/charts/  
weekly_sales_trend.png reports/charts/top_products.png reports/charts/category_decline.png reports/
data/weekly_metrics.csv logs/traces/run_xxx.json

15.4 周报报告结构

```
```markdown id="2lhdfa"
本周电商经营分析周报
```

## 1. 核心结论

## 2. 核心指标

- GMV
- 订单量
- 客单价
- 环比变化

## 3. Top 商品

```
4. Top 品类

5. 销售下降品类

6. 渠道表现

7. 数据支持结论

8. 需要进一步验证的假设

9. 下周建议

10. Trace 与 SQL 记录
```

## 15.5 周报功能定位

周报功能不是“LLM 写周报”，而是：

```
```text id="blbx4e" 多 Agent 拆解报告任务 多次调用 SQL 工具查询数据 调用图表工具生成可视化 调用
Evidence Validator 检查结论 调用 Report Tool 保存可追溯报告
```

```
---

### 15.6 行动型增强

P2 后半段加入 Action Workflow。

用户输入：

```text id="xn4lzm"
找出最近销售额下降最多的品类，并为运营团队创建跟进任务和监控告警。
```

流程：

```
```text id="hkkmki" Evidence Validator ↓ Action Planner ↓ Risk Assessor ↓ Approval Gate ↓ Action
Executor ↓ Action Verifier ↓ Audit Logger
```

```
行动工具：

```text id="2awzcg"
create_task()
create_metric_alert()
create_email_draft()
verify_action_execution()
log_audit_event()
```

审批规则：

```text id="c6pl9k" create\_task 需要审批 create\_metric\_alert 需要审批 create\_email\_draft 需要审批 导出  
大数据量需要审批 查询敏感字段需要审批或拒绝

16. P3 功能需求: MCP 与工程化

16.1 MCP Tool Layer

P3 再做 MCP, 不放进 MVP。

MCP Server 拆分:

```text id="oyjz5p"

database-mcp-server:  
 get\_database\_schema  
 get\_sample\_rows  
 run\_sql

report-mcp-server:  
 generate\_chart  
 save\_report

action-mcp-server:  
 create\_task  
 create\_metric\_alert  
 create\_email\_draft

不优先 MCP 化:

```text id="iwc3u" validate\_sql permission\_checker log\_trace eval\_runner

原因:

```text id="5c1s6z"

这些更适合作为系统内部安全、审计和评测模块。

---

## 16.2 Async Run API

P3 可加入:

```text id="5cmnin" POST /api/runs GET /api/runs/{run\_id} GET /api/runs/{run\_id}/trace GET /api/runs/  
{run_id}/events POST /api/runs/{run_id}/cancel

状态:

```
```text id="pqb80h"
queued
running
waiting_for_approval
completed
failed
cancelled
```

## 16.3 Trace Dashboard

Dashboard 展示:

```text id="b8juc7" Agent 节点耗时 工具调用次数 SQL 执行耗时 SQL 修复次数 失败类型分布 Eval 成功率  
Action 审批记录 Audit Log

```
---

## 17. LangGraph State 设计

### 17.1 P0 State

```python id="i0jl5i"
class AgentState(TypedDict):
 run_id: str
 session_id: str
 user_question: str

 database_schema: dict
 schema_text: str

 metric_context: dict
 selected_tables: list[str]

 generated_sql: str
 sql_reason: str
 review_result: dict

 execution_result: dict
 error_message: str
 fixed_sql: str
 retry_count: int

 final_answer: str
 trace: list[dict]
 status: str
```

## 17.2 P1 State 扩展

```
```python id="smsh65" business_context: dict evidence_result: dict python_analysis_result: dict
chart_paths: list[str] report_path: str
```

```
### 17.3 P2 State 扩展
```

```
```python id="khio57"
report_type: str
report_sections: list[dict]
action_plan: dict
approval_status: str
created_actions: list[dict]
audit_log_id: str
scheduled_followup: dict
```

---

## 18. 输出产物设计

### 18.1 P0 输出产物

每次 run 生成:

```
```text id="utavt6" generated_sql review_result execution_result fixed_sql, 可选 final_answer
trace.json eval/report.md
```

```
### 18.2 P1 输出产物
```

额外生成:

```
```text id="vmdh5a"
business_context.json
evidence_result.json
chart.png
report.md
```

### 18.3 P2 输出产物

额外生成:

```
```text id="wrw9za" weekly_business_report.md action_plan.json approval record tasks record
metric_alerts record action_verification_result.json audit_logs record
```

```
---
```

```
## 19. Eval 设计
```

Eval 必须从 P0 开始。

19.1 P0 Eval

准备 20 个测试问题。

覆盖：

```text id="4j4kae"

基础 SQL 查询

销售额排名

品类统计

时间趋势

销售下降分析

SQL 错误修复

危险 SQL 拦截

指标口径校验

指标：

```text id="uiydbm" SQL execution success rate SQL first-pass success rate SQL repair success rate  
dangerous SQL block rate metric definition accuracy average tool calls average latency failure type
distribution

目标：

```text id="fls3u9"

20 个测试问题中至少 15 个成功返回正确结果

危险 SQL 拦截率 100%

SQL 执行失败时不编造答案

所有失败都返回可解释原因

## 19.2 P1 Eval

新增：

```text id="u4jnr6" chart generation success rate report completeness evidence coverage rate  
unsupported claim rate

19.3 P2 Eval

新增：

```text id="qsv8aj"

weekly report completeness

approval trigger accuracy

task creation success rate



```
metric alert creation success rate
action verification success rate
audit log completeness
```

## 20. 安全需求

### 20.1 SQL 安全

必须实现：

```text id="596jtg" 只允许 SELECT 禁止 DROP / DELETE / UPDATE / INSERT / ALTER / TRUNCATE 禁止多语句执行 默认追加 LIMIT 限制最大返回行数 SQL timeout

20.2 敏感字段拦截

敏感字段：

```
```text id="xm8b9f"
phone
email
address
id_card
payment_info
```

如果用户查询敏感字段：

```text id="j617x0" SQL Reviewer 拒绝 不进入 run\_sql 记录 trace / audit 返回合规解释

20.3 高风险动作审批

P2 以后，以下动作需要审批：

```
```text id="t4m0eu"
create_task
create_metric_alert
create_email_draft
export_large_result
schedule_followup_run
```

## 21. 前端 Demo 需求

P0 使用 Streamlit。

页面模块：

```text id="f2hrap" 用户问题输入区 执行状态区 Agent 路由展示区 SQL 展示区 SQL Review 结果区 SQL 执行结果区 错误修复过程区 最终回答区 Trace 展示区 Eval 结果入口

P1 增加:

```text id="6pxc4w"

图表展示区

报告下载 / 查看区

Evidence 结果展示区

P2 增加:

```text id="xgdvnc" 周报生成入口 行动计划展示区 审批按钮 任务 / 告警创建结果区 Audit Log 展示区

22. 项目目录结构

22.1 P0 目录

```text id="q1ow3i"

insightflow-agent/

|—— README.md

|—— requirements.txt

|—— .env.example

|—— app.py

|

|—— data/

| |—— ecommerce.db

| |—— seed\_data.py

| |—— metrics.yaml

|

|—— agents/

| |—— supervisor.py

| |—— schema\_agent.py

| |—— metric\_agent.py

| |—— sql\_generator.py

| |—— sql\_reviewer.py

| |—— error\_fixer.py

| |—— insight\_agent.py

|

|—— tools/

| |—— schema\_tool.py

| |—— metric\_tool.py

| |—— sql\_validator.py

| |—— sql\_executor.py

| |—— trace\_logger.py

|

|—— graph/

```

| |—— state.py
| |—— nodes.py
| |—— workflow.py
|
|—— eval/
| |—— test_questions.json
| |—— run_eval.py
| |—— report.md
|
|—— logs/
| |—— traces/
|
|—— reports/
| |—— charts/
| |—— markdown/
|
|—— tests/
| |—— test_schema_tool.py
| |—— test_metric_tool.py
| |—— test_sql_validator.py
| |—— test_sql_executor.py
| |—— test_workflow.py
| |—— test_eval.py

```

## 22.2 P1 扩展目录

```

```text id="fsl0mq" agents/ |—— context_retriever.py |—— evidence_validator.py |——
chart_agent.py |—— report_agent.py

```

```

tools/ |—— context_tool.py |—— python_analysis_tool.py |—— chart_tool.py |—— report_tool.py

```

```

data/ |—— business_rules.md |—— table_docs.md |—— sql_examples.json

```

22.3 P2 扩展目录

```

```text id="kfp3qi"
agents/
|—— report_supervisor.py
|—— action_planner.py
|—— risk_assessor.py
|—— action_verifier.py

```

```

tools/
|—— action_tool.py
|—— approval_tool.py
|—— audit_logger.py

```

```

data 或 SQLite tables:
|—— reports

```

```
├── tasks
├── metric_alerts
├── approvals
├── audit_logs
└── scheduled_runs
```

## 22.4 P3 扩展目录

```
```text id="403i9z" mcp_servers/ ├── database_server.py ├── report_server.py └──
action_server.py
```

```
api/ └── main.py
```

```
dashboard/ Dockerfile docker-compose.yml .github/workflows/
```

```
---

## 23. 开发优先级

### P0: Agentic SQL Core

必须做:

```text id="r3du3f"
SQLite 电商数据库
Schema Tool
Metric Definition Tool
SQL Generator Agent
SQL Reviewer Agent
SQL Executor Tool
Error Fix Agent
Insight Agent
LangGraph Workflow
Trace Logger
20 个 Eval Case
Streamlit Demo
README
```

不做:

```
```text id="ohar2m" MCP 异步 Memory ActionOps RBAC Trace Dashboard 完整周报系统 复杂图表报告 多
数据源
```

```
---

### P1: Reliable Analysis & Report Core

新增:
```

```
```text id="54v8bd"
Business Context RAG 简化版
Evidence Validator
Data-supported findings vs hypotheses
run_python_analysis()
generate_chart()
save_report()
Markdown 分析报告
report eval
unsupported claim eval
```

---

## P2: Business Review & Action Workflow

新增:

```
```text id="9xcu40" 经营周报 / 品类复盘 / 渠道复盘 Report Supervisor Action Planner Risk Assessor
Approval Gate create_task() create_metric_alert() create_email_draft() Action Verifier Audit Log
ActionEval
```

```
---

### P3: MCP & Engineering Core

新增:

```text id="8rj0np"
MCP Tool Layer
FastAPI
Async Run
Trace Dashboard
Docker
CI
RBAC
Postgres
多数据源
```

---

## 24. 推荐 Demo

### P0 Demo 1: SQL 错误自动修复

用户问题:

```
```text id="8otuy7" 最近 30 天销售额最高的 5 个商品是什么?
```

展示链路：

```
```text id="9vgf67"
SQL Generator
-> run_sql failed: no such column
-> Error Fix Agent
-> validate_sql
-> run_sql success
-> Insight Agent
```

展示重点：

```text id="0z52rr" Action-Observation-Revision Execution Feedback Repair LangGraph Conditional Edge Tool Calling

```
---

### P0 Demo 2: Metric-aware SQL 查询

用户问题：

```text id="sxsdd6"
最近 30 天销售额最高的 5 个商品是什么？
```

展示链路：

```text id="t6jbxw" Metric Agent -> retrieve\_metric\_definition() SQL Generator 使用 GMV 口径 SQL Reviewer 检查 paid filter run\_sql 执行

展示重点：

```
```text id="t9u01d"
业务指标口径
不是只根据 Schema 生成 SQL
```

---

## P1 Demo：销售下降分析报告

用户问题：

```text id="v8e4zf" 最近 3 个月销售额下降最多的品类有哪些？生成趋势图和分析报告。

展示链路：

```
```text id="y7k35l"
```

SQL 查询  
Python 分析  
Evidence Validator  
Chart Agent  
Report Agent  
save\_report()

展示重点：

```text id="5g51wl" 图表报告不是 LLM 编的 结论可追溯到 SQL 和 execution\_result

P2 Demo：经营周报 + 行动计划

用户问题：

```text id="19geqq"  
帮我生成一份本周电商经营分析周报，并为下降品类创建跟进任务和监控告警。

展示链路：

```text id="70syqa" Report Supervisor 拆解周报 多个 SQL 查询 图表生成 Evidence 校验 保存周报 Action Planner Approval Gate create\_task create\_metric\_alert Audit Log

展示重点：

```text id="gwq06t"  
复杂多工具编排  
报告交付物  
行动型工具调用  
Human-in-the-loop

## 25. MVP 验收标准

P0 完成标准：

```text id="7nq2es" 用户可以通过 Streamlit 输入中文业务问题 系统可以创建 run\_id 系统可以调用 get\_database\_schema 获取真实 Schema 系统可以调用 retrieve\_metric\_definition 获取 GMV 等指标定义 SQL Generator 可以生成 SELECT SQL SQL Reviewer 可以调用 validate\_sql 校验 SQL 危险 SQL 会被拒绝，不进入 run\_sql SQL Executor 可以调用 run\_sql 真实执行 SQL SQL 执行失败后，Error Fix Agent 可以修复一次 修复后的 SQL 会重新经过 validate\_sql 和 run\_sql 最终回答基于 execution\_result，不编造 每次 run 都有完整 trace.json eval/run\_eval.py 可以运行 20 个测试问题 README 有启动说明、架构说明、Demo 示例和 Eval 结果

26. 简历写法

26.1 简洁版

****InsightFlow Agent | 基于 LangGraph 的多智能体工具调用式 BI Workflow****

基于 LangGraph 构建多智能体业务数据分析工作流，将 Schema 获取、业务指标定义、SQL 安全校验、SQL 执行和 Trace 记录封装为工具。系统支持自然语言业务问题到 SQL 生成、真实执行、错误反馈修复和结果解释的完整闭环，并通过 AgentEval 统计 SQL 成功率、修复成功率、危险 SQL 拦截率和工具调用指标。

26.2 技术版

设计 Schema Agent、Metric Agent、SQL Generator Agent、SQL Reviewer Agent、Error Fix Agent 和 Insight Agent 等多个专业 Agent，通过 LangGraph State 和条件边实现多步骤任务编排。系统将 `get\_database\_schema`、`retrieve\_metric\_definition`、`validate\_sql`、`run\_sql`、`log\_trace` 等能力封装为工具，使 Agent 能根据工具返回结果继续决策，并在 SQL 执行失败后进入 Action-Observation-Revision 修复流程。

26.3 报告增强版

实现 Report Agent，将多次 SQL 工具执行结果、图表路径、指标定义、数据支持结论与假设说明组合成可追溯的 Markdown 分析报告。报告中的核心结论均可回溯到 SQL、execution result 和 trace，避免 LLM 直接编造业务洞察。

26.4 行动增强版

后续引入 Action Planner、Risk Assessor、Approval Gate 和 Action Verifier，将分析结果转化为运营任务、指标告警和邮件草稿，并通过 Human-in-the-loop、Audit Log、MCP Tool Layer、异步 run 和 Trace Dashboard 提升系统的行动能力与工程化程度。

27. 最终项目目标

InsightFlow 的最终形态不是普通 ChatBI，而是：

```text id="rx3ord"

一个基于 LangGraph + MCP 的 Multi-Agent Tool-Calling BI Workflow。

最终闭环：

```text id="m63i70" Business Question -> Multi-Agent Planning -> Schema Tool -> Metric Tool -> SQL Generation -> SQL Review Tool -> SQL Execution Tool -> Error Repair -> Evidence Validation -> Chart -> Report -> Business Review -> Action Plan -> Approval -> Task / Alert -> Trace -> Eval


开发策略：

```text id="8a5gg4"

P0 先做到 Multi-Agent + Tool Calling + SQL Execution Feedback

P1 再做到 Reliable Analysis + Report Generation

P2 再做到 Business Review + Action Workflow

P3 最后做到 MCP + Engineering

项目第一原则：

text id="aevm83"

不要堆功能，要突出 Multi-Agent 和 Tool Calling。